

Artifacts and Evidence

Artifacts and Evidence I

This section reports what the project has actually produced and how each quantitative claim is grounded. Every number below is substituted at render time from the deterministic producer, which reads the repository state at commit `d09a9a6c3`, so each figure here is regenerated from the artifacts it describes rather than transcribed.

Model-Family Coverage I

GNN ships a curated corpus of model families that exercise the language across the difficulty gradient from minimal parser fixtures to full multi-agent and scaling studies. The manifest registers 9 families (basics, discrete, continuous, hierarchical, multiagent, precision, structured, gridworld, scaling-study) across 9 target directories, and `input/gnn_files` holds 11 corpus directories containing 30 concrete example models. The three sets are not coextensive, and the manuscript does not treat them as one.

Registered in no family: `input/gnn_files/learning`, `input/gnn_files/recursive` (2 of the 11 corpus directories), covered in sec. 7. Registered but outside the scanned tree: none, because all 9 target directories are themselves `input/gnn_files` corpus directories. Each family declares the simulation frameworks it is meant to drive, which is what lets the same

text model fan out across the executable-model leg of the Triple Play (Smékal and Friedman 2023). The families and their declared frameworks are enumerated below.

The exemplar corpus partitions cleanly by model kind, and the partition is itself a token: 27 of the 30 example specifications are discrete-state — the discrete categorical kind and its composed variants — and 3 are continuous linear-Gaussian specifications in the continuous family folder. The cold-start index `input/gnn_files/INDEX.md` catalogues every exemplar with its kind, its task folder, and a suggested reading order, and is the fastest surface for a reader who wants to move from this prose to a runnable file of a given kind.

The per-kind evidence is generated, not asserted: families that declare a capability axis (the continuous family declares the continuous-render axis) carry a backend split in tbl. 1 that is generated from the registry's own capability flags, so the table cannot disagree with the code that decides renderability. Reading the ledgers by kind is therefore mechanical — a continuous-family receipt shows the continuous-capable backends attempted and the categorical-only backends recorded unsupported, while a discrete-family receipt shows the categorical breadth the cross-framework gate anchors on (tbl. 4 is the per-kind key).

Model-Family Coverage I

Table 1: Model families declared in `input/model_family_manifest.json` with the simulation frameworks each family targets and a generated description. The *Frameworks* column is the family's declared targets, verbatim from the manifest; the capability splits appended to some descriptions are generated from `src/gnn/render/framework_registry.py` flags, not authored in the manifest, so they cannot disagree with the registry. Read with tbl. 3 (what each backend is) and fig. 1 (the same coverage as a matrix).

Family	Frameworks	Description
basics	pymdp	Minimal perception fixtures used for parser and validator smoke coverage.
discrete	pymdp	Discrete POMDP and HMM-style active inference fixtures.
continuous	jax, numpyro, stan, rxinfer	Continuous-state linear-Gaussian fixtures (F/H/Q/R + Gaussian prior; continuous_navigation closes the loop on beliefs). Native on RxInfer.jl, JAX, PyTorch, NumPyro, cpomdp, Stan; reported unsupported (not failed) on PyMDP, ActiveInference.jl, DisCoPy, bnlearn.
hierarchical	pymdp, rxinfer, jax	Hierarchical and temporal model fixtures (per-level matrices composed into a joint POMDP for categorical backends; RxInfer.jl renders two-level models natively).
multiagent	rxinfer	Multi-agent coordination and swarm fixtures.
precision	pymdp	Precision weighting and curiosity-driven fixtures.
structured	pymdp	Structured factor graph and posterior fixtures.
gridworld	pymdp, rxinfer, activeinference_jl	Gridworld POMDP fixture used for cross-framework acceptance checks.

Model-Family Coverage II

scaling-study

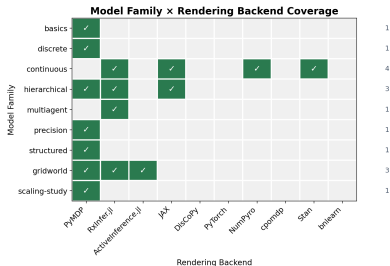
pymdp

PyMDP scaling-study fixtures,
sampled conservatively for
acceptance.

Model-Family Coverage I

The family-by-framework structure is shown in fig. 1, which renders the coverage matrix directly from the family registry rather than from a hand-maintained table.

Model-Family Coverage I



Declared coverage cells: 16 of 90 (9 families × 10 backends); the right-hand count is how many backends each family declares

Figure 1: Model-family coverage across the registered rendering backends: one row per family from `input/model_family_manifest.json`, one column per backend from `src/gnn/render/framework_registry.py`, and a green cell wherever the family declares that backend in its `frameworks` field. The right-hand count states how many backends each family declares; the grid is deliberately sparse — most families declare a single backend, and only continuous, hierarchical, and gridworld declare several. Read it as declared intent, not as profiled outcomes: the gates described below supply the outcomes. The matrix is generated from the two registries at commit d09a9a6c3.

Model-Family Coverage I

These families are not illustrative prose: they are the inputs over which the parser, the type checker, and the cross-framework code generators are exercised, and they are the substrate for the reliability gates described next.

Semantic-Fidelity and Cross-Framework Reliability Gates I

Two reproducible-by-command gates check that GNN's promise — one text model, many faithful executable renderings — survives contact with real backends. Both live under `scripts/` and read the same family corpus described above, so they verify the artifacts the manuscript actually references.

The semantic-fidelity gate, `scripts/run_semantic_fidelity_gate.py`, checks that a model parsed from GNN text and then re-emitted preserves its semantic content: the state-space structure, the factor and modality declarations, and the matrix shapes implied by a discrete Active Inference generative model survive the round trip (Da Costa et al. 2020). It is meant to be run as a command and to report fidelity per model, not as a static claim baked into prose.

Semantic-Fidelity and Cross-Framework Reliability Gates I

The cross-framework reliability gate, `scripts/run_cross_framework_reliability.py`, takes a single GNN model and renders it across multiple simulation backends, then checks that the resulting executable models agree on the structure they were generated from. The reference comparison runs on the continuous family across JAX, NumPyro, RxInfer.jl — 3 independent Active Inference engines spanning the Python and Julia ecosystems (Heins et al. 2022). The family declares 4 backends (JAX, NumPyro, Stan, RxInfer.jl); Stan is declared but not among the 7 frameworks the gate profiles, so it is excluded from the comparison. Because the same source model drives all 3 renderings, disagreement between backends localizes a generator defect rather than a modeling choice.

Semantic-Fidelity and Cross-Framework Reliability Gates I

Both gates are stated here as commands you can run, not as asserted pass counts. The manuscript deliberately does not quote a fixed number of passing checks: the authoritative, current result is whatever those scripts report when executed against the corpus, and binding a frozen count into prose would invite exactly the drift the auto-injection contract exists to prevent.

A third interchange check extends the same discipline across repositories: `scripts/run_geo_interchange_checks.py` validates the committed pin in `.github/gnn-pair.json` against a selected GEO-INFER checkout and replays exported GNN artifacts — the tracked gridworld, a compiled H3 stay/diffuse model, a rectangular Gaussian, and an explicit factored fixture — inside the GEO environment, writing its receipts even on failure. Like the gates above it is stated as a command, not as a pass count, and its pinned pair

Semantic-Fidelity and Cross-Framework Reliability Gates I

(ActiveInferenceInstitute/GEO-INFER at a recorded revision)
is what makes “interchange” a checkable claim rather than a
promise.

Repository Scale I

The repository's scale is itself evidence of the surface that the gates and pipeline cover, and it is reported in fig. 2 directly from the tracked files at commit d09a9a6c3.

Repository Scale I

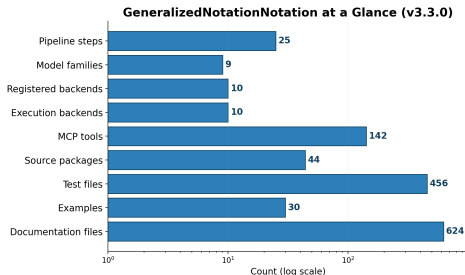


Figure 2: Repository-scale counts on a logarithmic axis: pipeline steps, model families, registered backends, execution backends, Model Context Protocol tools, source packages, test files, example models, and documentation files. Every bar is annotated with its exact value, and every value is a producer token read from `output/data/manual/script_variables.json` at commit `d09a9a6c3` — the same token map that substitutes the prose counts, so the figure cannot disagree with the text without failing the figure-freshness suite. The two backend bars are deliberately distinct: *registered backends* (10) counts render targets, *execution backends* (10) counts the subset that runs at Step 12. Read the chart as the scale of the surface the pipeline maintains, not as a quality measure.

The test suite comprises 456 test files containing 4900 test functions, exercising a source base of 683 Python files across 44 packages (198544 lines of source). The pipeline's step modules — the thin orchestrators named in tbl. 2 — number 25, one per numbered step. The Model Context Protocol surface — which exposes GNN's capabilities to external agents and tools — provides 142 tools across 32 modules. The pipeline itself runs as 25 steps (0–24), and 7 figure artifacts from the rendering of figures, models, and reports are committed under output/, of which 7 are the manuscript's own.

A claim is manuscript-ready only when it is bound to a verifiable artifact. Concretely, every claim in this manuscript must rest on one of four support types:

Claim Discipline I

- ▶ A passing test or validator command — for example the semantic-fidelity and cross-framework gates above, which can be re-run on demand.
- ▶ A generated output produced by a deterministic producer, such as the figures rendered from the family and repository scans, or the token values emitted by the manuscript-variable producer that backs every number on this page.
- ▶ A source ledger, manifest, or configuration file that fixes the value being claimed.
- ▶ A resolved entry in `references.bib` for any external-literature claim (Friston 2010; Parr et al. 2022).

Claim Discipline I

The pipeline records its own evidence trail under `output/`. The run-level summary is written to `output/PIPELINE_REPORT.md`, and the per-step execution record — including which steps ran, their status, and their artifacts — is captured in `output/00_pipeline_summary`. These are the artifacts a reader should consult to confirm that the numbers substituted into this section correspond to a real, reproducible pipeline run rather than to asserted prose. When a value would otherwise need a literal number with no producer behind it, the discipline is to omit the number rather than to hard-code it.